Calidad y Preprocesamiento de Datos

MCIC VÍCTOR MANUEL CORZA VARGAS

MC FERNANDO AVITÚA VARELA

Unidad 5 Transformación y Preprocesamiento de Datos

5.1.5 Transformación numérica (min max, z-score, logarítmica, binning)

Introducción

Las variables numéricas a menudo requieren transformaciones para ajustar su escala, modificar su distribución o cambiar su representación. Estas transformaciones son cruciales para que muchos algoritmos de Machine Learning funcionen correctamente, para mejorar la interpretabilidad de los modelos o para manejar características específicas de los datos, como el sesgo (skewness) o la presencia de outliers.

Escalado Min-Max (Normalización) y Estandarización Z-score:

(Recapitulación de 5.1.1): Como vimos anteriormente, estas son las técnicas de escalado más comunes.

Min-Max: Transforma los datos a un rango fijo, usualmente [0, 1].

$$x' = \frac{x - \min(X)}{\max(X) - \min(X)}$$

Uso: Útil cuando se necesita un rango acotado; sensible a outliers.

Z-score: Transforma los datos para tener media 0 y desviación estándar 1.

$$x' = \frac{x - \mu}{\sigma}$$

Uso: Método estándar para muchos algoritmos (regresión lineal, SVM, PCA); menos sensible a outliers que Min-Max.

Transformación Logarítmica (Log Transform)

Qué es: Aplicar la función logaritmo (natural ln(x) o base 10 $\log_{10}(x)$) a cada valor de la característica.

Por qué: Es particularmente útil para manejar datos con sesgo positivo (right-skewed), es decir, distribuciones con una cola larga hacia los valores altos (ej. ingresos, precios de casas, conteos).

La transformación logarítmica comprime el rango de los valores grandes y expande el de los pequeños, haciendo la distribución más simétrica y cercana a la normal. Esto beneficia a modelos lineales que asumen normalidad de errores/residuos.

Transformación Logarítmica (Log Transform)

La transformación logarítmica comprime el rango de los valores grandes y expande el de los pequeños, haciendo la distribución más simétrica y cercana a la normal. Esto beneficia a modelos lineales que asumen normalidad de errores/residuos.

Fórmula: $x' = \log(x)$ (se asume $x > 0$).

Consideraciones: La variable original debe ser estrictamente positiva. Si hay ceros o valores negativos, se suele sumar una constante pequeña antes de aplicar el logaritmo (ej. $\log(x+1)$).

Transformación Logarítmica (Log Transform)

Ejemplo: Datos originales (sesgados): [10, 50, 100, 1000, 10000].

Datos con $\log_{10}$: [1, 1.7, 2, 3, 4].

La diferencia entre 1000 y 10000 (9000) se reduce a una diferencia de 1 (de 3 a 4) en la escala logarítmica.

Binning (Discretización)

(Relacionado con 5.1.4): Consiste en convertir una variable numérica continua en una variable categórica discreta, agrupando los valores en "bins" o intervalos.

Por qué:

- Puede hacer que los modelos sean más robustos a pequeños cambios en los datos y a outliers (que caen en el primer o último bin).
- Permite capturar relaciones no lineales de forma simple si los bins se tratan como categorías (ej. en árboles de decisión).
- Simplifica la variable para ciertos tipos de análisis o algoritmos. (Por ejemplo, naive bayes)

Binning (Discretización)

Métodos Comunes de Binning:

- **Equal Width (Ancho Fijo):** Se divide el rango total de la variable (max - min) en k intervalos del mismo tamaño. Fácil de implementar, pero si la distribución es sesgada, algunos bins pueden quedar casi vacíos y otros muy llenos.
- **Equal Frequency (Frecuencia Fija / Cuantiles):** Se divide la variable en k intervalos de forma que cada intervalo contenga aproximadamente el mismo número de observaciones (100/k percentiles). Maneja mejor distribuciones sesgadas, pero los anchos de los intervalos variarán.
- **Manual/Basado en Dominio:** Se definen los puntos de corte de los intervalos según el conocimiento experto (ej. grupos de edad estándar: 0-17, 18-25, 26-40, 41-65, 65+).

Binning (Discretización)

Ejemplo: Variable 'Edad': [21, 24, 29, 35, 42, 48, 55, 62].

- *Binning de Ancho Fijo (k=3):*

  Rango = 62-21 = 41.

  Ancho ≈ 41/3 ≈ 13.7.

  Bins: [21, 34.7), [34.7, 48.4), [48.4, 62].

Ejemplo Binning por Frecuencia (Equal Frequency / Quantile Binning)

Imagina que tenemos los siguientes datos de 'Puntuacion_Examen' para 12 estudiantes:

Puntuaciones = [55, 62, 65, 68, 70, 72, 75, 78, 82, 88, 95, 98]

Queremos dividir estas puntuaciones en **k = 4 bins** usando el método de frecuencia fija(cuartiles). Esto significa que cada bin debería contener el mismo número de estudiantes, que es 12/4=3 estudiantes por bin.

Ejemplo Binning por Frecuencia (Equal Frequency / Quantile Binning)

Paso 1 Ordenar los datos: Los datos ya están ordenados. [55, 62, 65, 68, 70, 72, 75, 78, 82, 88, 95, 98]

Paso 2 Determinar los puntos de corte: Buscamos los valores que dividen los datos en 4 grupos de 3 estudiantes cada uno.

- **Grupo 1 (Primer Cuartil - Q1):** Contiene los 3 valores más bajos: [55, 62, 65]. El valor más alto de este grupo es 65.
- **Grupo 2 (Segundo Cuartil - Q2):** Contiene los siguientes 3 valores: [68, 70, 72]. El valor más alto de este grupo es 72.
- **Grupo 3 (Tercer Cuartil - Q3):** Contiene los siguientes 3 valores: [75, 78, 82]. El valor más alto de este grupo es 82.
- **Grupo 4 (Cuarto Cuartil - Q4):** Contiene los últimos 3 valores: [88, 95, 98]. El valor más alto de este grupo es 98.

Ejemplo Binning por Frecuencia (Equal Frequency / Quantile Binning)

Definir los Bins (Intervalos): Los puntos de corte son 65, 72 y 82. Los bins se definen (usando la notación (min,max] o similar):

- **Bin 1 (Bajo):** $(-\infty,65]$ (Contiene 55, 62, 65)
- **Bin 2 (Medio-Bajo):** (65,72] (Contiene 68, 70, 72)
- **Bin 3 (Medio-Alto):** (72,82] (Contiene 75, 78, 82)
- **Bin 4 (Alto):** $(82,\infty)$ o (82,98] (Contiene 88, 95, 98)

Ejemplo Binning por Frecuencia (Equal Frequency / Quantile Binning)

Resultado de la Transformación: Cada puntuación original se reemplaza por la etiqueta de su bin:

Puntuación Original	Bin (Etiqueta)
55	Bajo
62	Bajo
65	Bajo
68	Medio-Bajo
70	Medio-Bajo
72	Medio-Bajo
75	Medio-Alto
78	Medio-Alto
82	Medio-Alto
88	Alto
95	Alto
98	Alto

Ejemplo Binning por Frecuencia (Equal Frequency / Quantile Binning)

Observación Clave: Cada bin ('Bajo', 'Medio-Bajo', 'Medio-Alto', 'Alto') contiene exactamente 3 observaciones. Sin embargo, el *rango numérico* de cada bin es diferente:

- Bin 1: Cubre implícitamente hasta 65.
- Bin 2: Cubre $72-65=7$ puntos.
- Bin 3: Cubre $82-72=10$ puntos.
- Bin 4: Cubre $98-82=16$ puntos.

Esto contrasta con el binning de ancho fijo, donde los rangos numéricos serían iguales, pero el número de observaciones por bin podría variar mucho si los datos no están distribuidos uniformemente.

Conclusión 5.1.5 Transformación numérica (min max, z-score, logarítmica, binning)

La elección de la transformación numérica adecuada (escalado, logaritmo, binning) depende de las características de la distribución de los datos, la presencia de outliers, los requisitos del algoritmo a utilizar y los objetivos del análisis. A menudo se experimenta con varias transformaciones para ver cuál funciona mejor.

Actividad de clase

1. Formar equipos de 5 personas

2. Crear un prompt bien estructurado para un LLM, que solicite generar una libreta de jupyter con un ejemplo sintético sencillo de clasificación en el que se realice alguna transformación numérica de las que hemos abordado hasta el momento, solicitar al LLM que incluya un explicación clara de cada paso desarrollado.

Transformaciones numéricas:

- **Min-max (1 equipo)**
- **Z-score (1 equipo)**
- **Logarítmica (2 equipos)**
- **Binning (2 equipos)**

3. Cada equipo contará con 5 minutos para realizar una exposición rápida del proceso generado.

5.1.6 Transformación categórica (one-hot, label encoding, hashing, Nominal, frecuencia)

Introducción: Las variables categóricas representan cualidades o etiquetas (ej. colores, países, tipos de cliente). La mayoría de los algoritmos de Machine Learning requieren entradas numéricas, por lo que necesitamos convertir estas categorías en números de una manera que sea significativa y útil para el algoritmo.

Distinguimos principalmente entre variables **Nominales** (categorías sin orden inherente, como 'Rojo', 'Verde', 'Azul') y **Ordinales** (categorías con un orden lógico, como 'Bajo', 'Medio', 'Alto'). Las técnicas que veremos se centran sobre todo en las variables nominales.

Métodos Principales de Transformación Categórica

Label Encoding (Codificación de Etiquetas)

Qué es: Asigna un número entero único a cada categoría. Ej: {'Rojo': 0, 'Verde': 1, 'Azul': 2}.

Por qué: Es simple y no aumenta el número de columnas.

Cuándo usar (con precaución):

- **Para variables Ordinales:** Si los números asignados respetan el orden natural (ej. {'Bajo': 0, 'Medio': 1, 'Alto': 2}).
- **Para variables Nominales en modelos basados en árboles (Decisión Trees, Random Forest):** Estos modelos pueden manejar números enteros sin interpretar necesariamente un orden, aunque a veces One-Hot puede dar mejores resultados.
- **Para variables binarias (dos categorías):** Ej: {'Sí': 0, 'No': 1}.

Label Encoding (Codificación de Etiquetas)

Desventajas: Para variables nominales, introduce un **orden artificial** que no existe (ej. Azul > Verde > Rojo), lo cual puede confundir a algoritmos sensibles a la magnitud o distancia (modelos lineales, SVM, k-NN).

Ejemplo: Variable 'Mascota': {'Gato': 0, 'Perro': 1, 'Pez': 2}.

One-Hot Encoding

Qué es: Crea una nueva columna binaria (0 o 1) para cada categoría presente en la variable original. Para cada observación, la columna correspondiente a su categoría tendrá un 1 y las demás tendrán un 0.

Por qué: Es el método estándar y generalmente **recomendado para variables nominales** porque no introduce un orden artificial. Representa claramente la presencia o ausencia de una categoría.

One-Hot Encoding

Desventajas:

- Aumenta significativamente el número de columnas si la variable tiene muchas categorías (alta cardinalidad), lo que puede llevar a problemas de dimensionalidad y rendimiento.
- Puede introducir multicolinealidad perfecta (una columna se puede predecir a partir de las otras). A menudo se soluciona eliminando una de las columnas (drop='first').

Ejemplo: Variable 'Ciudad': {'Madrid', 'París', 'Roma'}. Crea columnas Ciudad_Madrid, Ciudad_Paris, Ciudad_Roma. Una observación con 'París' se codifica como [0, 1, 0].

One Hot Coding Vs Dummy Coding

Column	One Hot Code	Dummy Code
Blackberry	100	10
Grape	010	01
Orange	001	00

Dummy Encoding: similar to one hot encoding. While one hot encoding utilises N binary variables for N categories in a variable. Dummy encoding uses N-1 features to represent N labels/categories

Hashing Encoder (Codificación por Hashing)

Qué es: Utiliza una función hash para convertir el nombre de la categoría en un índice numérico. Mapea las categorías a un número fijo y predefinido de columnas de salida.

Por qué: Controla la dimensionalidad de salida, independientemente de cuántas categorías existen. Muy útil para **variables de alta cardinalidad** (ej. códigos postales) o en entornos de streaming donde pueden aparecer nuevas categorías. Es eficiente en memoria.

Hashing Encoder (Codificación por Hashing)

Desventajas:

- Colisiones: Diferentes categorías pueden ser mapeadas a la misma columna de salida (colisión hash), lo que implica pérdida de información.
- Las columnas resultantes no son directamente interpretables (no sabemos qué categoría original representa cada columna hash).

Ejemplo Hashing Encoder con Colisión

Supongamos que tenemos una característica categórica 'Ciudad' y queremos codificarla usando un Hashing Encoder con un **número fijo de características de salida = 5**.

Nuestras categorías son: 'Londres', 'Paris', 'Tokio', 'Nueva York', 'Berlin', 'Sidney'.

Paso 1

Elegir una Función Hash: Para simplificar, usaremos una función hash muy básica: sumar los valores ASCII de los caracteres y luego aplicar el módulo 5 (el número de características de salida). *Nota: Las implementaciones reales usan funciones hash más robustas.*

Ejemplo Hashing Encoder con Colisión

Paso 2 Aplicar la Función Hash a cada Categoría:

'Londres': L(76)+o(111)+n(110)+d(100)+r(114)+e(101)+s(115) = 727. 727(mod5)=2. Índice de salida: **2**.

'Paris': P(80)+a(97)+r(114)+i(105)+s(115) = 511. 511(mod5)=1. Índice de salida: **1**.

'Tokio': T(84)+o(111)+k(107)+i(105)+o(111) = 518. 518(mod5)=3. Índice de salida: **3**.

'Nueva York': N(78)+u(117)+e(101)+v(118)+a(97)+ ( )+Y(89)+o(111)+r(114)+k(107) = 932 (ignorando espacio). 932(mod5)=2. Índice de salida: **2**. <-- **¡COLISIÓN!** ('Londres' también dio índice 2).

'Berlin': B(66)+e(101)+r(114)+l(108)+i(105)+n(110) = 604. 604(mod5)=4. Índice de salida: **4**.

'Sidney': S(83)+i(105)+d(100)+n(110)+e(101)+y(121) = 620. 620(mod5)=0. Índice de salida: **0**.

Ejemplo Hashing Encoder con Colisión

Representación Resultante (Formato One-Hot sobre índices hash): Creamos 5 columnas (índices 0 a 4). Para cada ciudad, ponemos un '1' en la columna correspondiente a su índice hash.

Ciudad Original	Hash Mod 5	Feature_0	Feature_1	Feature_2	Feature_3	Feature_4
Londres	2	0	0	1	0	0
Paris	1	0	1	0	0	0
Tokio	3	0	0	0	1	0
Nueva York	2	0	0	1	0	0
Berlin	4	0	0	0	0	1
Sidney	0	1	0	0	0	0

Ejemplo Hashing Encoder con Colisión

Tanto 'Londres' como 'Nueva York' resultaron en el índice hash **2**. Esto significa que en la representación final, ambas ciudades tienen exactamente el mismo vector de características: [0, 0, 1, 0, 0].

Ejemplo Hashing Encoder con Colisión

Consecuencia: Si alimentamos estas características a un modelo de Machine Learning, el modelo no tendrá forma de distinguir entre 'Londres' y 'Nueva York' basándose únicamente en esta característica transformada. La información que las distinguía se ha perdido debido a la colisión hash.

Ejemplo Hashing Encoder con Colisión

Este es el compromiso (trade-off) que se hace al usar Hashing para limitar la dimensionalidad: se gana eficiencia y control sobre el número de características, pero se corre el riesgo de perder información debido a las colisiones. *Nota: Algunas variantes de hashing usan signos (+1/-1) para intentar mitigar el efecto de las colisiones, pero la ambigüedad fundamental persiste.*

Referencias

Han, J., Pei, J., & Tong, H. (2022). Data mining: concepts and techniques. Morgan kaufmann.

Ponniah, P. (2004). Data warehousing fundamentals: a comprehensive guide for IT professionals. John Wiley & Sons.

Scikit-learn documentation. Preprocessing data https://scikit-learn.org/stable/modules/preprocessing.html